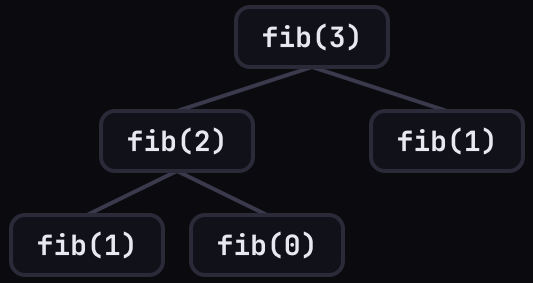
// T-OLIMPIANS — PREZENTACJA DO LEKCJI

Drzewo rekurencji i memoizacja

Co się dzieje, gdy funkcja woła samą siebie dwa razy: drzewo wywołań, kolejność ich wykonywania i to, dlaczego potrafi być gigantyczne. Potem zeszyt z wynikami — memoizacja — i zadania „na ile sposobów?".

C++ · REKURENCJA · CZĘŚĆ 3 Z 6 · PO CZĘŚCIACH 1-2



PLAN PREZENTACJI

01 Dwa wywołania naraz
02 Ciąg Fibonacciego
03 Drzewo wywołań
04 Kolejność wywołań
05 Ile jest wywołań?
06 Memoizacja
07 Schody
08 Ścieżki na planszy
09 Trójkąt Pascala
10 Najlepszy wynik: żaba
11 Wieże Hanoi
12 Dwa parametry
13 Sprawdź się
14 Podsumowanie

01 Przypomnienie: funkcja, która woła siebie dwa razy

```
void linijka(int n) {
    if (n == 0) return;
    linijka(n - 1);
    cout << n << " ";
    linijka(n - 1);
}
```

LINIJKA(3) WYPISUJE

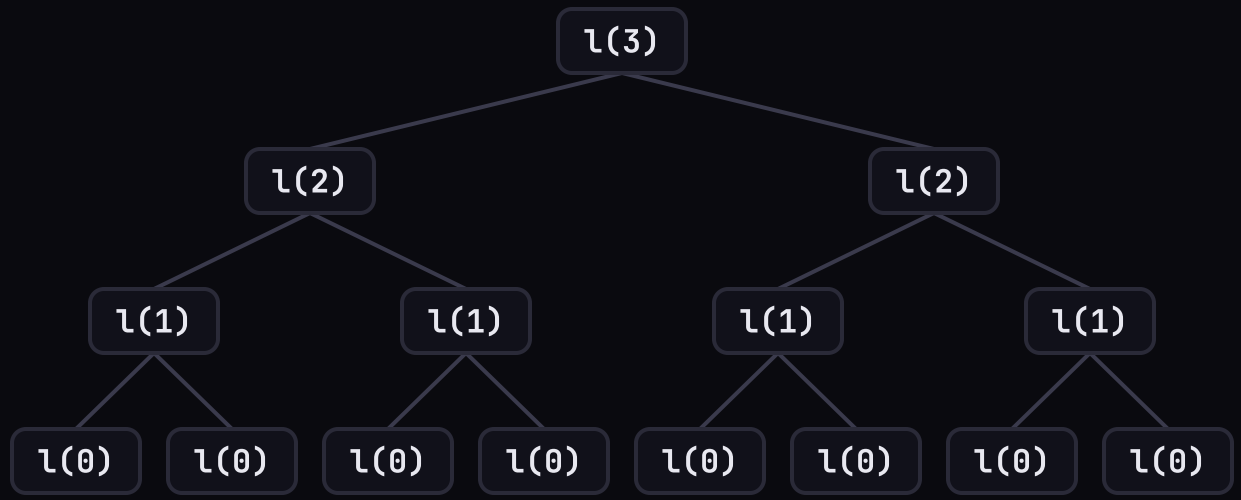
```
1 2 1 3 1 2 1
```

To pytanie 5 z części 1. Funkcja ma **dwa** wywołania samej siebie: jedno przed wypisaniem, drugie po nim.

`linijka(3)` = całe `linijka(2)` → **3** → całe `linijka(2)`.

> Gdy funkcja woła siebie więcej niż raz, wywołania nie układają się już w **linię**, tylko rozgałęziają się jak **drzewo**.

01 Drzewo wywołań `linijka(3)`



Każdy prostokąt to **jedno wywołanie** (l = linijka). Strzałki w dół to wywołania: każde l(n) z n > 0 woła **dwa** razy l(n − 1). Na samym dole — warunki stopu l(0).

01 Ile razy wywoła się `linijka(n)`?

n	wywołań	wypisanych liczb
1	3	1
2	7	3
3	15	7
10	2 047	1 023
20	ok. 2 milionów	ok. miliona
30	ok. 2 miliardów	ok. miliarda

Każdy poziom drzewa ma **dwa razy więcej** wywołań niż poprzedni: 1, 2, 4, 8, … Razem dla n jest ich $\mathbf{2^{n+1} - 1}$.

Uwaga: Drzewo rośnie **wykładniczo**. Dla linijki to nie problem — i tak trzeba wypisać $2^n - 1$ liczb. Ale za chwilę zobaczymy funkcję, w której takie drzewo to czysta strata czasu.

02 Ciąg Fibonacciego

Każdy wyraz to **suma dwóch poprzednich**. Zaczynamy od 0 i 1:

F	0	1	1	2	3	5	8	13	21	34	55
indeks	F(0)	F(1)	F(2)	F(3)	F(4)	F(5)	F(6)	F(7)	F(8)	F(9)	F(10)

$$F(0) = 0,\ F(1) = 1,\ F(n) = F(n-1) + F(n-2)$$

Na przykład F(7) = F(6) + F(5) = 8 + 5 = 13. Ten ciąg pojawia się w zadaniach olimpijskich zaskakująco często — za chwilę zobaczysz go w schodach i ciągach bez sąsiednich jedynek.

02 Fibonacci rekurencyjnie — prosto z definicji

```
int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
```

FIB(7)

13

Pytanie	Odpowiedź
Kiedy kończę?	n ≤ 1: fib(0) = 0, fib(1) = 1 — czyli **return n**
Jak zmniejszyć?	**dwa** mniejsze problemy: `fib(n - 1)` i `fib(n - 2)`
Co zwracam?	sumę obu wyników

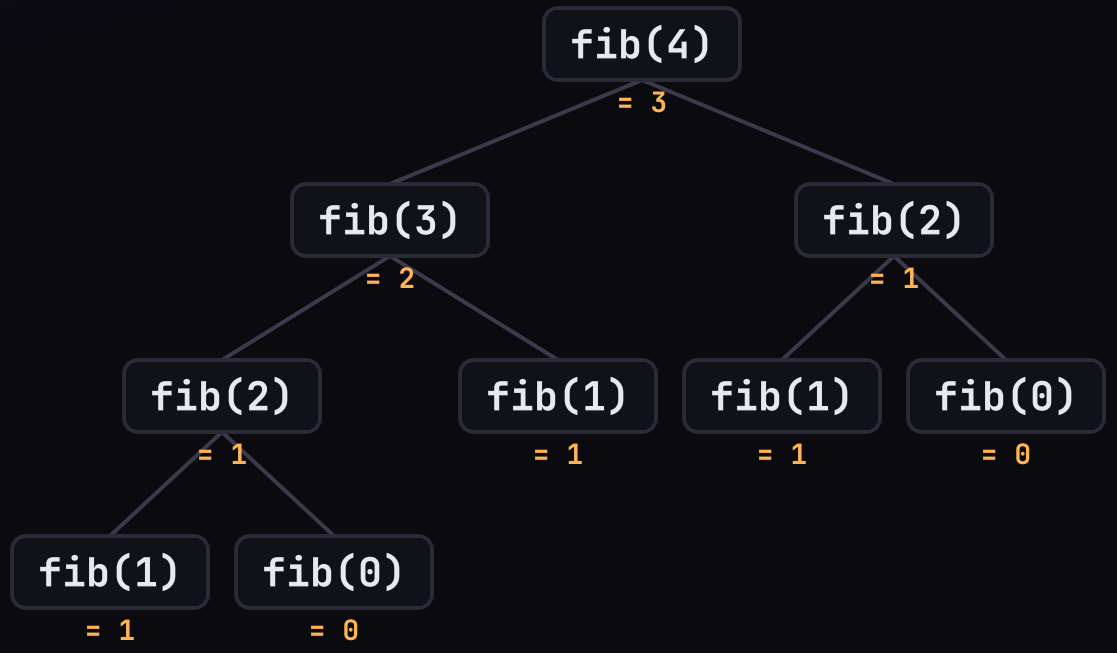
02 Linijka z dwoma wywołaniami

```
return fib(n - 1) + fib(n - 2);
       ①    ②        ③
```

1	`return`	oddaj sumę temu, kto wywołał
2	`fib(n - 1)`	liczy się pierwsze — CAŁE, ze wszystkimi swoimi wywołaniami
3	`fib(n - 2)`	liczy się drugie — dopiero gdy pierwsze się skończy

Kopia fib(n) czeka na stosie **dwa razy**: najpierw na wynik fib(n − 1), potem na wynik fib(n − 2). Dopiero wtedy dodaje i zwraca.

03 Drzewo wywołań `fib(4)`

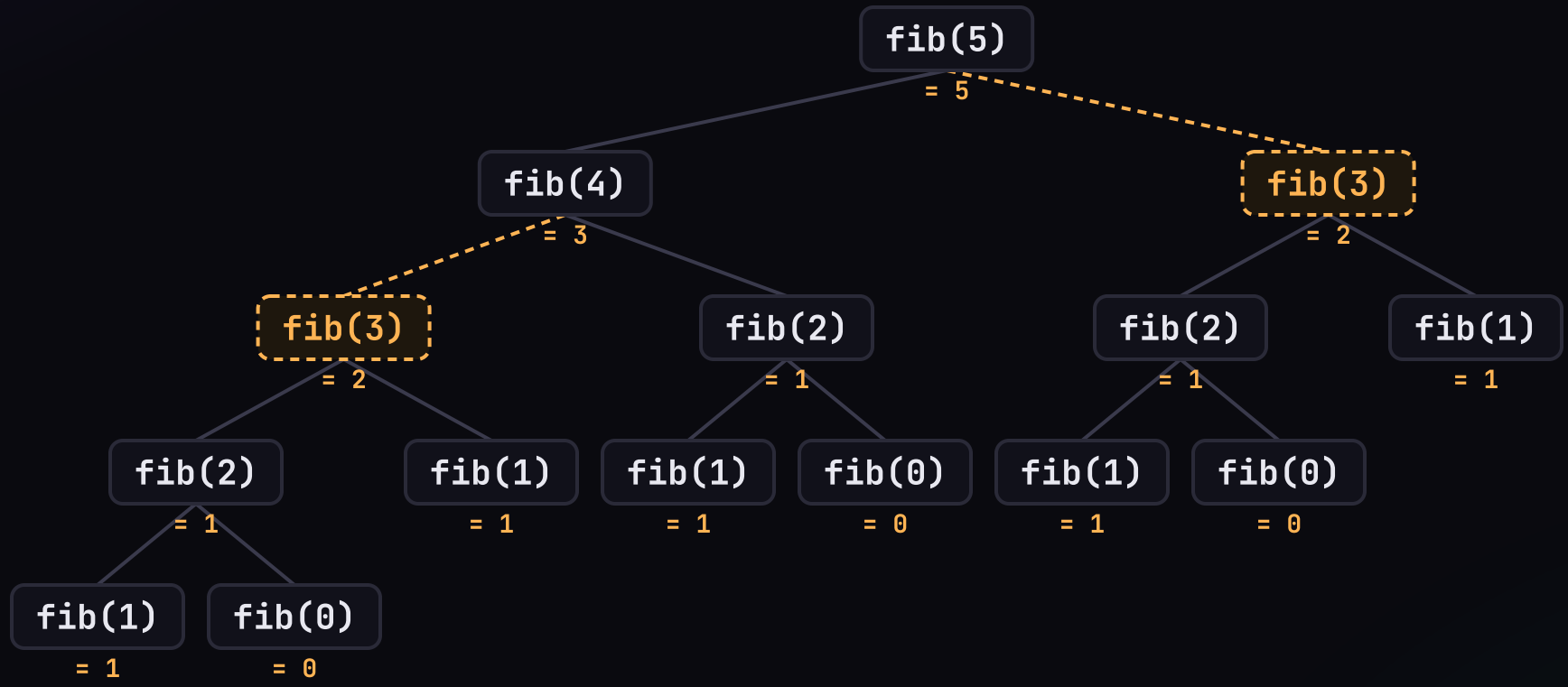


Pod każdym wywołaniem — wartość, którą zwraca (pomarańczowa).

1. fib(4) woła fib(3) i fib(2).
2. fib(3) woła fib(2) i fib(1).
3. Każde fib(2) woła fib(1) i fib(0).
4. fib(1) i fib(0) to **liście** — warunki stopu.

> Wartości liczy się **od liści w górę**: 1 + 0 = 1, potem 1 + 1 = 2, i tak dalej, aż na górze wyjdzie 3.

03 Drzewo wywołań `fib(5)`



Zauważ: **fib(3)** liczy się tu **dwa razy** — i za każdym razem schodzi przez całe swoje poddrzewo, choć wynik jest ten sam: 2.

03 Te same obliczenia wiele razy

Wywołanie	ile razy w fib(5)	ile razy w fib(10)
`fib(5)`	1	—
`fib(4)`	1	—
`fib(3)`	2	21
`fib(2)`	3	34
`fib(1)`	5	55
`fib(0)`	3	34

Przy większym n jest jeszcze gorzej. W drzewie fib(10) samo fib(1) pojawia się 55 razy, a fib(2) — 34 razy.

Problem nie jest w rekurencji, tylko w liczeniu tego samego wiele razy.

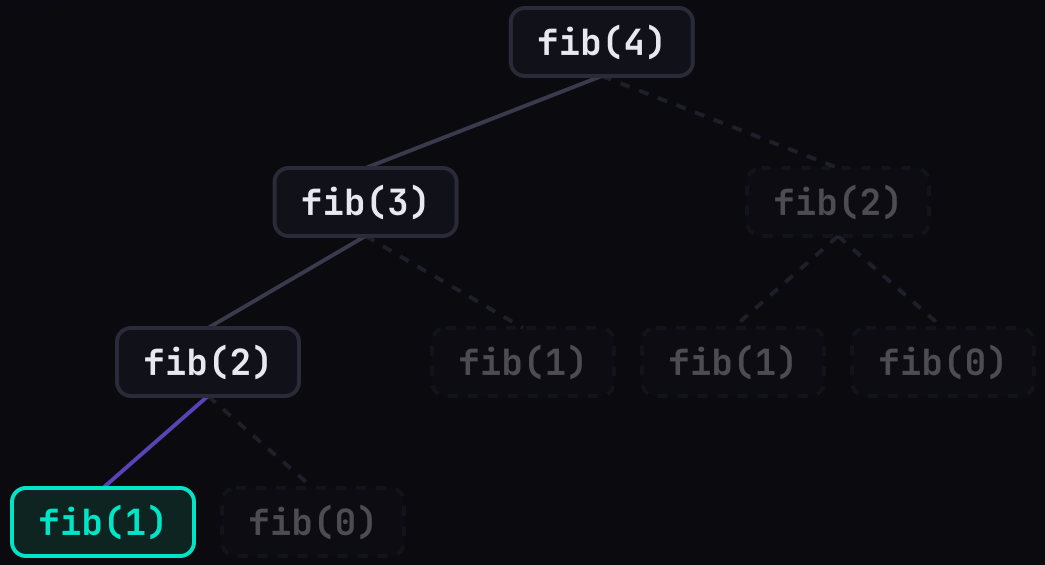
04 W jakiej kolejności wykonują się wywołania?

Komputer schodzi najpierw w lewo, do samego dna, a dopiero po powrocie idzie w prawo.

W `return fib(n - 1) + fib(n - 2);` najpierw liczy się całe **fib(n − 1)** — z całym jego poddrzewem. Dopiero gdy ono zwróci wynik, zaczyna się **fib(n − 2)**.

> Na kolejnych slajdach: **turkusowe** — wywołanie, które właśnie działa, białe — czekające na stosie, **fioletowe** — już skończone (z wynikiem), wyblakłe — jeszcze się nie zaczęły.

04 Klatka 1: schodzimy w lewo do `fib(1)`



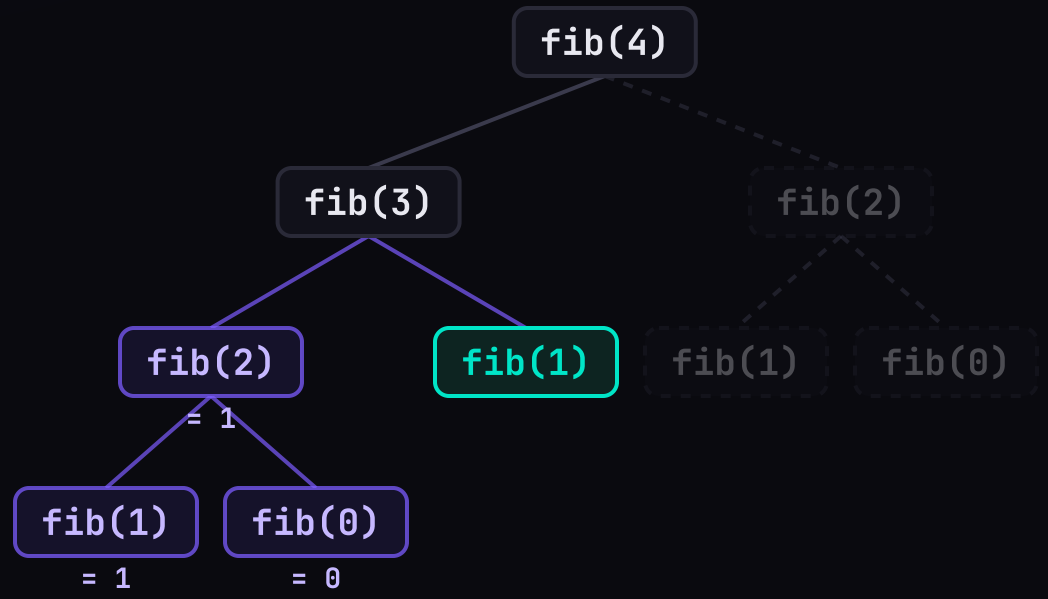
fib(4) → fib(3) → fib(2) → fib(1). Na stosie leżą cztery kopie fib, każda czeka na swoje **lewe** dziecko.

STOS WYWOŁAŃ

- `fib(1)` działa
- `fib(2)` czeka
- `fib(3)` czeka
- `fib(4)` czeka
- `main()` czeka

DNO STOSU

04 Klatka 2: `fib(2)` gotowe, teraz prawe dziecko `fib(3)`



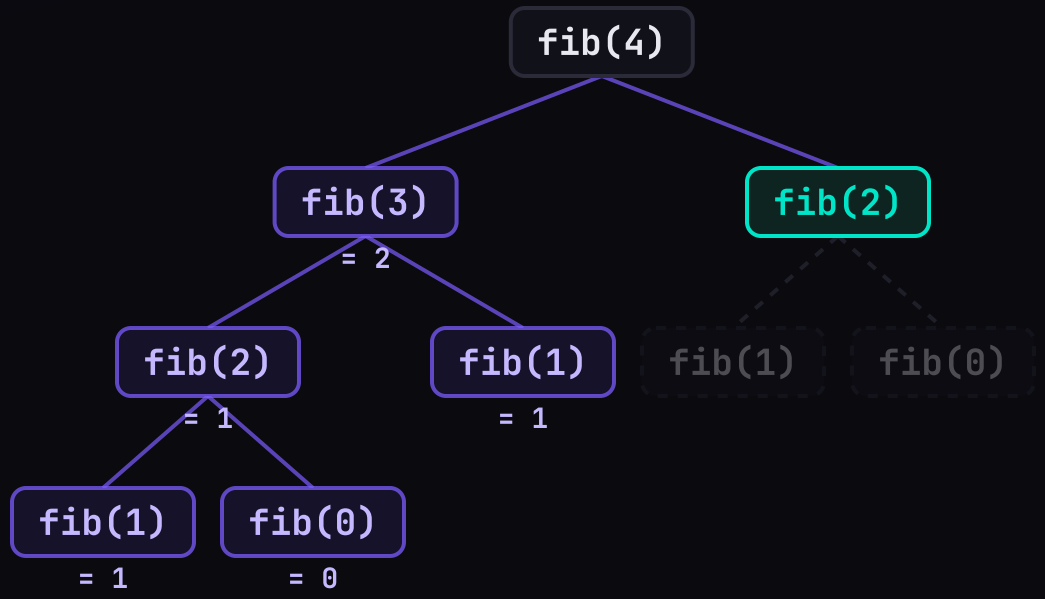
fib(1) zwróciło 1, fib(0) zwróciło 0, więc fib(2) = 1 i zniknęło ze stosu. fib(3) dostało pierwszy wynik i woła **prawe** dziecko: fib(1).

STOS WYWOŁAŃ

- `fib(1)` działa
- `fib(3)` czeka
- `fib(4)` czeka
- `main()` czeka

DNO STOSU

04 Klatka 3: `fib(3)` = 2, zaczyna się prawa połowa



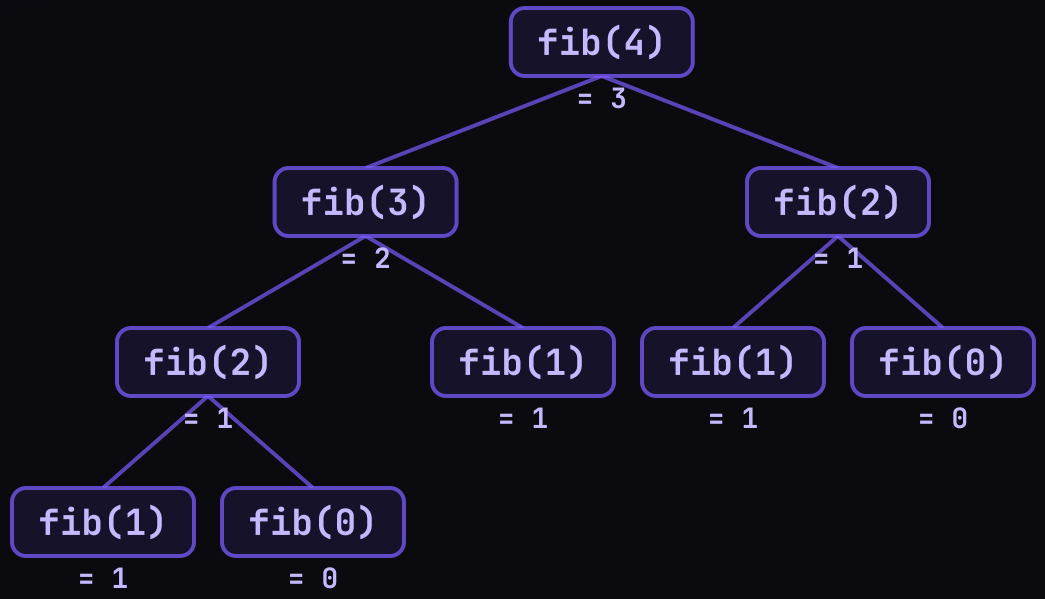
fib(3) = 1 + 1 = 2. Teraz fib(4) ma wynik lewego dziecka i woła prawe: **fib(2)** — które znowu zejdzie do fib(1) i fib(0).

STOS WYWOŁAŃ

- `fib(2)` działa
- `fib(4)` czeka
- `main()` czeka

DNO STOSU

04 Klatka 4: wszystko policzone



Prawe fib(2) = 1, więc fib(4) = 2 + 1 = 3 — i wynik wraca do main. Każde wywołanie zostało odwiedzone dokładnie raz, w kolejności: najpierw lewe poddrzewo, potem prawe.

STOS WYWOŁAŃ

`main()` czeka

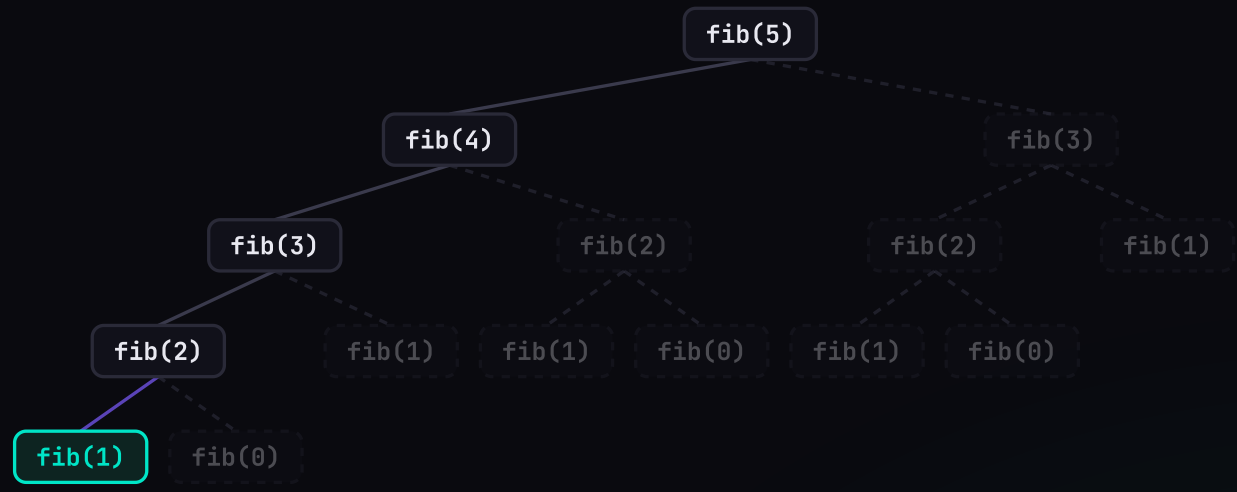
DNO STOSU

04 Stos jest tak wysoki jak drzewo — nie większy

W każdej chwili na stosie leżą tylko wywołania na **drodze** od korzenia do wywołania, które właśnie działa.

Drzewo fib(n) ma wysokość n, więc stos ma najwyżej **n** pięter — nawet gdy wszystkich wywołań są miliony.

> Pamięć to nie problem. Problemem jest **czas**: każde z milionów wywołań trzeba wykonać.



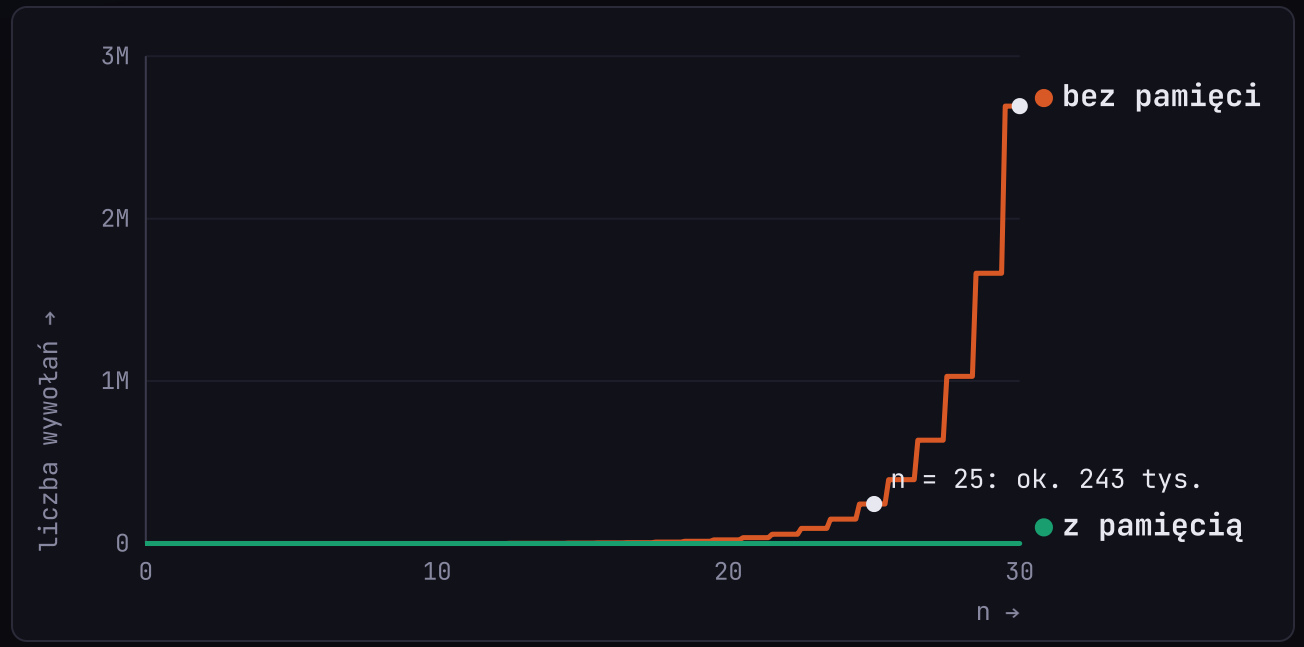
05 Ile wywołań wykona `fib(n)`?

n	F(n)	wywołań fib	czas
10	55	177	błyskawicznie
20	6 765	21 891	błyskawicznie
30	832 040	ok. 2,7 mln	ok. 0,01 s
40	102 334 155	ok. 331 mln	**ok. 1 s**
50	ok. $1{,}3 \cdot 10^{10}$	ok. $4 \cdot 10^{10}$	**ok. 2 minuty**
90	ok. $2{,}9 \cdot 10^{18}$	ok. $9 \cdot 10^{18}$	**setki lat**

Liczbę wywołań W(n) łatwo policzyć — też rekurencyjnie: **W(n) = 1 + W(n − 1) + W(n − 2)**, W(0) = W(1) = 1.

> Wychodzi, że W(n) = 2 · F(n + 1) − 1. Wywołań jest mniej więcej **tyle, ile wynosi sam wynik** — a F(n) rośnie prawie 1,6 raza z każdym krokiem.

05 Wykres: wywołania rosną wykładniczo



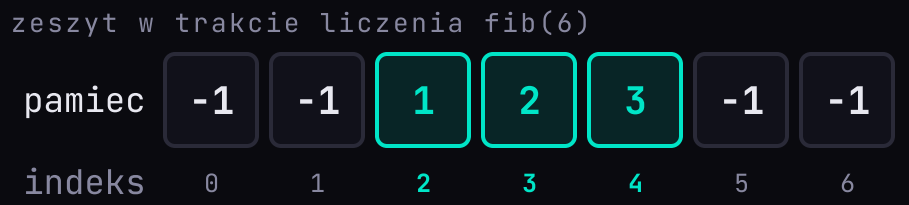
Pomarańczowa linia stoi „płasko”, a potem nagle wystrzela w górę — to znak funkcji **wykładniczej**.

Zielona linia to ta sama funkcja z **pamięcią** (za chwilę): dla n = 30 tylko **59** wywołań — na wykresie nawet nie odrywa się od osi.

06 Pomysł: zapisuj wyniki w zeszycie

Zanim zaczniesz liczyć, zajrzyj do zeszytu. Gdy policzysz — zapisz wynik.

Zeszyt to zwykła tablica **pamiec[]**: w komórce pamiec[n] trzymamy policzone już fib(n). Na start wpisujemy wszędzie **-1**, co znaczy „jeszcze nie liczone”.



> Ten sposób ma swoją nazwę: **memoizacja** (od angielskiego *memo* — notatka). To pierwszy krok do programowania dynamicznego.

06 Fibonacci z pamięcią

```
long long pamiec[91];

long long fib(int n) {
    if (n <= 1) return n;
    if (pamiec[n] != -1) return pamiec[n];
    pamiec[n] = fib(n - 1) + fib(n - 2);
    return pamiec[n];
}

int main() {
    for (int i = 0; i <= 90; i++) pamiec[i] = -1;
    cout << fib(90);
}
```

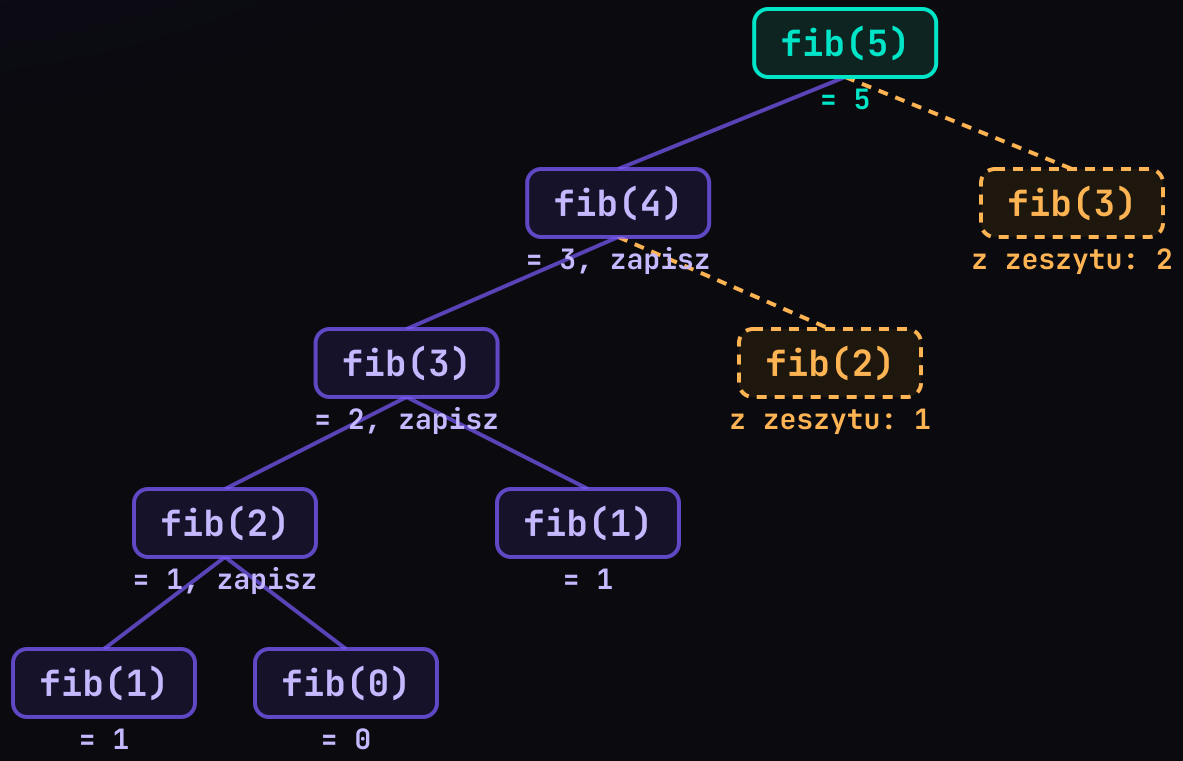
1. linijka 5: **sprawdź zeszyt** — jeśli fib(n) już było liczone, zwróć zapisany wynik od razu,
2. linijka 6: w przeciwnym razie **policz** tak jak dawniej i **zapisz** w zeszycie,
3. linijka 7: zwróć wynik.

FIB(90) — NATYCHMIAST

```
2880067194370816120
```

Zeszyt wypełniamy −1 w main. **−1** nadaje się na znacznik, bo prawdziwe fib(n) nigdy nie jest ujemne.

06 Drzewo fib(5) z pamięcią



Gdy drugi raz potrzebujemy fib(2) i fib(3), są już w zeszycie (**pomarańczowe**). Całe ich poddrzewa **znikają**.

	bez pamięci	z pamięcią
fib(5)	15 wywołań	**9**
fib(30)	ok. 2,7 mln	**59**
fib(90)	ok. $9 \cdot 10^{18}$	**179**

06 Memoizacja — przepis na każdą funkcję

```
long long pamiec[MAX];          // MAX — większe niż największy argument

long long f(int n) {
    if (warunek stopu) return wynik;
    if (pamiec[n] != -1) return pamiec[n];   // 1. zajrzyj do zeszytu
    pamiec[n] = ... f(mniejsze) ...;         // 2. policz i zapisz
    return pamiec[n];                        // 3. zwróć
}
```

kiedy pomaga?

Gdy te same wywołania (z tymi samymi argumentami) powtarzają się w drzewie **wiele razy**.

kiedy nie pomaga?

Gdy każde wywołanie jest inne — np. odliczanie albo wypisywanie wszystkich ciągów (część 4).

ile kosztuje?

Tyle wywołań, ile jest **różnych argumentów** — każdy liczony raz — i tablica na wyniki.

07 Schody

Bajtek wchodzi na schody o **n** stopniach. Jednym krokiem pokonuje **1** albo **2** stopnie. Na ile sposobów może wejść na szczyt?

n	sposoby	ile
1	1	1
2	1+1, 2	2
3	1+1+1, 1+2, 2+1	3
4	1+1+1+1, 1+1+2, 1+2+1, 2+1+1, 2+2	5

Popatrz na **ostatni krok**. Na stopień n Bajtek wszedł:

1. ze stopnia **n − 1** (krokiem o 1) — sposobów jest tyle, ile dojść do n − 1,
2. albo ze stopnia **n − 2** (krokiem o 2) — tyle, ile dojść do n − 2.

sposoby(n) = sposoby(n − 1) + sposoby(n − 2)

07 Schody — kod

```
long long pamiec[81];

long long sposoby(int n) {
    if (n <= 1) return 1;
    if (pamiec[n] != -1) return pamiec[n];
    pamiec[n] = sposoby(n - 1) + sposoby(n - 2);
    return pamiec[n];
}
```

Warunki stopu: **sposoby(1) = 1** (jeden krok o 1) i **sposoby(0) = 1** — gdy już stoimy na szczycie, jest dokładnie jeden sposób: nic nie robić.

To dokładnie Fibonacci, tylko przesunięty o jeden: sposoby(n) = F(n + 1). Bez pamięci byłby tak samo wolny jak fib.

Uwaga: Wynik dla n = 80 to ok. $3{,}8 \cdot 10^{16}$ — **long long**.

07 A gdyby kroki mogły mieć długość od 1 do k?

```
long long sposoby(int n) {
    if (n == 0) return 1;
    if (pamiec[n] != -1) return pamiec[n];
    long long wynik = 0;
    for (int d = 1; d <= k && d <= n; d++)
        wynik += sposoby(n - d);
    return pamiec[n] = wynik;
}
```

Ostatni krok mógł mieć długość 1, 2, …, k. Sumujemy więc **k** wywołań — w pętli. Pętla w funkcji rekurencyjnej to nic złego!

`return pamiec[n] = wynik;` — przypisanie i zwrócenie w jednej linijce: najpierw zapisujemy, a potem zwracamy zapisaną wartość.

08 Ścieżki na planszy

Pionek idzie z lewego górnego rogu do prawego dolnego, tylko **w prawo** albo **w dół**. Ile jest ścieżek?

Na pole (i, j) pionek mógł wejść tylko **z góry** albo **z lewej**. Więc liczba ścieżek do pola = ścieżki do pola nad nim + ścieżki do pola po lewej.

$$ś(i, j) = ś(i - 1, j) + ś(i, j - 1)$$

1	1	1	1	1
1	2	3	4	5
1	3	6	10	15
1	4	10	20	35

6 = 3 (z góry) + 3 (z lewej). W pierwszym wierszu i pierwszej kolumnie jest tylko **1** ścieżka — prosto.

08 Ścieżki — kod z pamięcią 2D

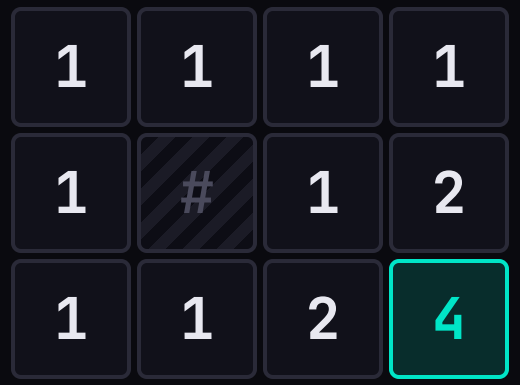
```
long long pamiec[18][18];

long long sciezki(int i, int j) {
    if (i == 1 || j == 1) return 1;
    if (pamiec[i][j] != -1) return pamiec[i][j];
    pamiec[i][j] = sciezki(i - 1, j) + sciezki(i, j - 1);
    return pamiec[i][j];
}
```

Funkcja ma **dwa parametry**, więc zeszyt jest **dwuwymiarowy**: pamiec[i][j] to wynik dla pola (i, j).

plansza	bez pamięci	z pamięcią
4 × 5	ok. 70 wywołań	≤ 20 pól
17 × 17	**ponad miliard**	**≤ 289 pól**

08 Ścieżki z przeszkodami



Na zablokowane pole (#) prowadzi **0** ścieżek. Wystarczy jeden dodatkowy warunek stopu:

```
long long sciezki(int i, int j) {
    if (i < 0 || j < 0 || g[i][j] == '#') return 0;
    if (i == 0 && j == 0) return 1;
    ...
}
```

> Tu pola numerujemy od 0, a start (0, 0) ma jedną ścieżkę. Wyjście poza planszę też daje 0 — dzięki temu pierwszy wiersz i kolumna liczą się same.

09 Symbol Newtona: na ile sposobów wybrać k z n?

C(n, k) — na ile sposobów można wybrać **k** osób z grupy **n** osób. Np. z 4 osób parę można wybrać na C(4, 2) = 6 sposobów.

Spójrz na **jedną osobę** — Bajtka:

1. wybieramy Bajtka — dobieramy jeszcze k - 1 osób z n - 1 pozostałych: **C(n – 1, k – 1)**,
2. nie wybieramy go — wszystkie k osób z n - 1 pozostałych: **C(n – 1, k)**.

$$C(n, k) = C(n - 1, k - 1) + C(n - 1, k)$$

Warunki stopu: C(n, 0) = 1 (nikogo nie wybieramy) i C(n, n) = 1 (wybieramy wszystkich).

09 Trójkąt Pascala

n	C(n, 0) … C(n, n)
0	1
1	1 1
2	1 2 1
3	1 3 3 1
4	1 4 **6** 4 1
5	1 5 10 10 5 1
6	1 6 15 20 15 6 1

Każda liczba to suma **dwóch liczb nad nią**: 6 = 3 + 3. To jest właśnie nasz wzór rekurencyjny.

```
long long C(int n, int k) {
    if (k == 0 || k == n) return 1;
    if (pamiec[n][k] != -1) return pamiec[n][k];
    return pamiec[n][k] = C(n - 1, k - 1) + C(n - 1, k);
}
```

10 Żaba na kamieniach

Żaba skacze z kamienia 1 na kamień n. Z kamienia i skacze na **i + 1** albo **i + 2**. Skok kosztuje **|różnica wysokości|**. Jaki jest najmniejszy łączny koszt?

h	10	30	40	20
indeks	1	2	3	4

Tym razem **nie liczymy sposobów**, tylko szukamy **najlepszego**. Na kamień i żaba przyskoczyła z i − 1 albo z i − 2 — wybieramy tańszą możliwość:

$$\text{koszt}(i) = \min(\ \text{koszt}(i-1) + |h_i - h_{i-1}|,\ \text{koszt}(i-2) + |h_i - h_{i-2}|\ )$$

10 Żaba — liczymy po kolei

i	z i - 1	z i - 2	koszt(i)
1	—	—	**0** (start)
2	0 + |30 − 10| = 20	—	**20**
3	20 + |40 − 30| = 30	0 + |40 − 10| = 30	**30**
4	30 + |20 − 40| = 50	20 + |20 − 30| = 30	**30**

```
long long koszt(int i) {
    if (i == 1) return 0;
    if (pamiec[i] != -1) return pamiec[i];
    long long w = koszt(i - 1) + abs(h[i] - h[i - 1]);
    if (i >= 3)
        w = min(w, koszt(i - 2) + abs(h[i] - h[i - 2]));
    return pamiec[i] = w;
}
```

Liczba sposobów → **dodajemy** wyniki wywołań.
Najlepszy wynik → **min albo max** z wyników.

11 Wieże Hanoi

Trzy pręty, na pierwszym n krążków od największego do najmniejszego. Przenieś wszystko na pręt 3:

1. w jednym ruchu przenosisz **jeden** krążek z wierzchu,
2. nigdy nie kładziesz **większego na mniejszym**.

Jak przenieść n krążków z A na C? Rekurencyjnie!

1. **przenieś n − 1 górnych krążków z A na B (używając C jako pomocniczego),**
2. **przełóż największy krążek z A na C,**
3. **przenieś n − 1 krążków z B na C (używając A jako pomocniczego).**

11 Hanoi — kod

```
void hanoi(int n, int z, int na, int przez) {
    if (n == 0) return;
    hanoi(n - 1, z, przez, na);
    cout << z << " -> " << na << endl;
    hanoi(n - 1, przez, na, z);
}
```

Z main: `hanoi(3, 1, 3, 2)` — 3 krążki z pręta 1 na pręt 3, przez pręt 2.

HANOI(3, 1, 3, 2)

```
1 -> 3
1 -> 2
3 -> 2
1 -> 3
2 -> 1
2 -> 3
1 -> 3
```

11 Ile ruchów?

n	ruchów
1	1
2	3
3	7
10	1 023
20	ok. miliona
64	ok. $1{,}8 \cdot 10^{19}$

ruchy(n) = ruchy(n − 1) + 1 + ruchy(n − 1) = **2 · ruchy(n − 1) + 1**, a ruchy(0) = 0. Wychodzi $\mathbf{2^n - 1}$.

> Tu memoizacja **nic nie da**: hanoi(n − 1, …) za każdym razem przenosi krążki **między innymi prętami**, więc każdy ruch jest inny i trzeba go wypisać.

Legenda mówi, że mnisi przenoszą 64 krążki od początku świata. Przy jednym ruchu na sekundę skończą za ok. 585 miliardów lat.

12 Rozkłady liczby na składniki

Na ile sposobów można zapisać n jako sumę dodatnich liczb, jeśli kolejność nie ma znaczenia? Dla n = 4: **4, 3+1, 2+2, 2+1+1, 1+1+1+1** — 5 sposobów.

R(n, k) = liczba rozkładów n na składniki **nie większe niż k**. Dzielimy rozkłady na dwie grupy:

1. **bez** składnika k → R(n, k − 1),
2. **z** co najmniej jednym składnikiem k → zabieramy go: R(n − k, k).

```
long long R(int n, int k) {
    if (n == 0) return 1;
    if (k == 0) return 0;
    if (k > n) return R(n, n);
    if (pamiec[n][k] != -1) return pamiec[n][k];
    return pamiec[n][k] = R(n, k - 1) + R(n - k, k);
}
```

Odpowiedź: R(n, n). Dla n = 100 to ponad 190 milionów rozkładów — policzonych w ułamku sekundy.

12 Monety: na ile sposobów wydać resztę?

Nominały $c_1, \ldots, c_n$, każdego nieograniczenie wiele. Na ile sposobów wypłacić kwotę S? Ten sam pomysł co w rozkładach — patrzymy na **ostatni nominał**:

1. nie używamy go wcale → sposoby(i − 1, s),
2. używamy go choć raz → odkładamy jedną monetę: sposoby(i, s − c_i).

```
long long sposoby(int i, int s) {
    if (s == 0) return 1;
    if (s < 0 || i == 0) return 0;
    if (pamiec[i][s] != -1) return pamiec[i][s];
    return pamiec[i][s] =
        sposoby(i - 1, s) + sposoby(i, s - c[i]);
}
```

Dla monet 1, 2, 5 i kwoty 5: 5 · 1+2+2 · 1+1+1+2 · 1+1+1+1+1 — 4 sposoby. Dwa parametry → zeszyt dwuwymiarowy.

13 Pytanie 1: ile wywołań wykona `f(4)`?

```
int f(int n) {
    if (n == 0) return 1;
    return f(n - 1) + f(n - 1);
}
```

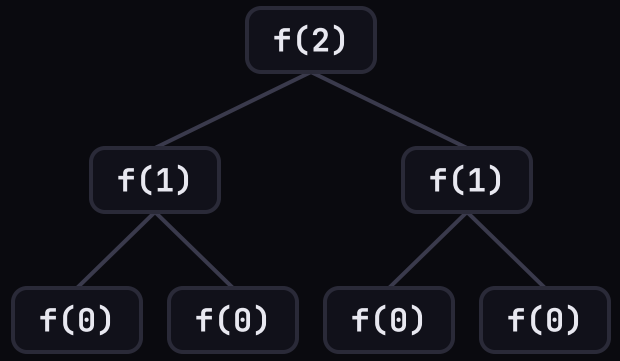
NA EKRANIE

?

A ile wynosi wynik?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

13 Odpowiedź 1: 31 wywołań, wynik 16



Obok drzewo f(2): 7 wywołań. Każde wywołanie woła dwa razy to samo, więc drzewo f(n) ma $1 + 2 + 4 + \ldots + 2^n = \mathbf{2^{n+1} - 1}$ wywołań: dla n = 4 to **31**. Wynik to $2^n = \mathbf{16}$.

> Wystarczy policzyć raz: `int x = f(n - 1); return x + x;` — tylko **n + 1** wywołań!

13 Pytanie 2: ile wywołań wykona `fib(10)` z pamięcią?

```
long long pamiec[91];

long long fib(int n) {
    if (n <= 1) return n;
    if (pamiec[n] != -1) return pamiec[n];
    pamiec[n] = fib(n - 1) + fib(n - 2);
    return pamiec[n];
}
```

NA EKRANIE

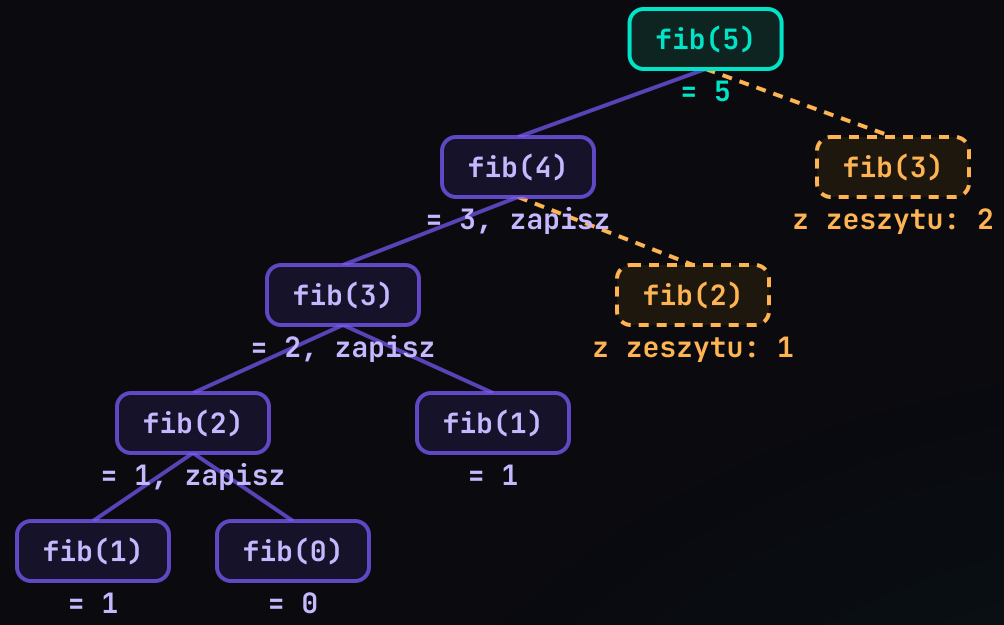
?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

13 Odpowiedź 2: 19

fib(n) woła fib(n − 1) — pełne wywołanie — i fib(n − 2), które jest już w zeszycie (od razu zwraca). Wywołania idą więc po „lewej krawędzi” drzewa i każde ma jedno szybkie prawe dziecko.

	liczba wywołań
fib(10) z pamięcią	$2 \cdot 10 - 1 = \mathbf{19}$
fib(10) bez pamięci	**177**



13 Pytanie 3: ile wynosi `sciezki(3, 3)`?

```
long long sciezki(int i, int j) {
    if (i == 1 || j == 1) return 1;
    return sciezki(i - 1, j) + sciezki(i, j - 1);
}
```

NA EKRANIE

?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

13 Odpowiedź 3: 6

1	1	1
1	2	3
1	3	6

Każde pole = pole nad nim + pole po lewej. Na planszy 3 × 3 jest **6** ścieżek: PPDD, PDPD, PDDP, DPPD, DPDP, DDPP.

13 Pytanie 4: dlaczego to wciąż jest wolne?

```
long long fib(int n) {
    if (n <= 1) return n;
    if (pamiec[n] != -1) return pamiec[n];
    return fib(n - 1) + fib(n - 2);
}
```

NA EKRANIE

?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

13 Odpowiedź 4: nic nie zapisujemy!

POPRAWNIE

```
long long fib(int n) {
    if (n <= 1) return n;
    if (pamiec[n] != -1) return pamiec[n];
    return pamiec[n] = fib(n - 1) + fib(n - 2);
}
```

Uwaga: Funkcja zagląda do zeszytu, ale **nigdy do niego nie pisze** — wszędzie zostaje -1, więc liczy wszystko od nowa. Zawsze sprawdź obie części: **czytanie i zapisywanie**.

14 Ściąga: drzewo rekurencji i memoizacja

Pojęcie	Co to znaczy	Pamiętaj
drzewo wywołań	każde wywołanie to węzeł, jego wywołania to dzieci	liście = warunki stopu
kolejność	najpierw całe lewe poddrzewo, potem prawe	stos = droga od korzenia
liczba wywołań	przy dwóch wywołaniach rośnie jak 2^n	policz, zanim uruchomisz
memoizacja	zeszyt pamiec[] z -1 na start: sprawdź, policz, zapisz	wywołań tyle, ile różnych argumentów
liczba sposobów	dodajemy wyniki wywołań	schody, ścieżki, Pascal, monety
najlepszy wynik	min / max z wyników wywołań	żaba
dwa parametry	zeszyt dwuwymiarowy pamiec[i][j]	ścieżki, C(n, k), rozkłady

14 Zadania na koniec — Twoja kolej

Zadania są na **t-olimpians.com** → **Kompendium** → **Funkcje rekurencyjne** → **Krok 3**. Każde ma sprawdzarkę i wskazówkę:

1. Liczby Fibonacciego
2. Linijka
3. Wieże Hanoi
4. Fibonacci z pamięcią
5. Ile wywołań?
6. Schody
7. Schody z dłuższymi krokami
8. Trójkąt Pascala
9. Ścieżki na planszy
10. Ścieżki z przeszkodami
11. Żaba na kamieniach
12. Na ile sposobów rozłożyć liczbę?
13. Ile sposobów wydania reszty?

Memoizacja to wstęp do **programowania dynamicznego** — następny temat w kompendium to „Memoizacja i tabulacja”.

// KONIEC CZĘŚCI 3

Drzewo nie straszy, gdy masz zeszyt!

Dwa wywołania tworzą drzewo, a drzewo rośnie wykładniczo. Gdy te same wywołania się powtarzają, zapisuj wyniki — memoizacja zamienia miliardy wywołań w kilkadziesiąt. Zrób zadania z kroku 3, a potem część 4: generowanie wszystkich możliwości.

